Language Generation in the Limit

Fabio Colletti

October 2026

Chapter 1

Introduction

La tesi si basa sulla descrizione e discussione dei risultati dell'articolo Language Generation in the Limit di Jon Kleinberg e Sendhil Mullainathan.

1.1 Notazione e Definizioni

In questa sezione vediamo quali sono le notazioni utilizzate nel corso dell'elaborato, e le definizioni non banali.

Definition 1. *Si definisce* ***Alfabeto*** *un insieme* Σ *di caratteri.*

Esempi di Alfabeti sono l'insieme dei numeri naturali $\mathbb{N} := \{1, 2, 3, \dots\}$,
o l'insieme delle lettere dell'alfabeto $A = \{a, b, c, \dots, z\}$

Definition 2. *Si definisce* ***Universo delle parole*** Σ^* *l'insieme di tutte le possibili stringhe di caratteri* Σ *compresa la stringa vuota* ϵ.

Ogni stringa (o parola) è un elemento di Σ^*.

Note 1. *L'operazione che permette il passaggio dall'alfabeto* Σ *all'universo delle parole* Σ^* *è nota come* ***Chiusura di Kleene****.*

Definition 3. *Un* ***Linguaggio*** L *è un sottoinsieme di qualche* Σ^*. *In particolare un linguaggio può rispondere o meno a delle regole.*

Example 1. *(Esempio di alfabeto, universo, linguaggio, e parole)*

Sia Σ *un alfabeto.* $\Sigma := \{0, 1\}$.

Sia Σ^* *l'universo delle parole generato da* Σ, *ossia* $\Sigma^* := \{\epsilon, 0, 1, 01, 111, 1010, 0000000, 10101, \dots\}$.

Sia L *il linguaggio formato solo dalle stringhe che iniziano con zero, ossia* $L := \{0, 01, 00, 0101, \dots\}$

*La stringa "*1010*" è un elemento di* Σ^*, *ma non è un elemento di* L.

Note 2. *In seguito, gli universi delle parole* Σ^* *saranno sempre considerati* ***numerabili****.*

1.2 Modello Formale

In questa sezione verrà descritto formalmente il modello utilizzato.

Sia dato un alfabeto Σ ed il suo universo delle parole (numerabile) Σ^*.
Da questo, si estrae una lista numerabile di linguaggi $C = \{L_1, L_2, \dots\}$.
Sia ogni linguaggio $L_i,\ i \in \mathbb{N}$ infinito e numerabile.

Ha senso osservare che i linguaggi si possono ripetere.

Un avversario ed un algoritmo giocano secondo la seguente dinamica: in principio l'avversario fissa un linguaggio in C, che denotiamo con K. L'algoritmo non è a conoscenza del linguaggio scelto.
A questo punto l'avversario inizia ad enunciare le stringhe di K una ad una secondo una sequenza $t = [1, 2, \dots]$. In questa enumerazione ci possono essere ripetizioni, l'importante è che per ogni parola $w \in K$ esiste almeno un "tempo" t in cui verrà enunciata.
Sia w_i la parola enunciata al tempo $t = i$.

Definiamo S_t come l'insieme delle parole che l'avversario ha enunciato dal tempo inziale $t' = 1$ al tempo $t' = t$, ossia $S_t := \{w_1, w_2, \dots, w_t\}$

In questo modello possiamo ora specificare il Problema di identificazione (problema di Gold-Angluin) e il Problema di Generazione

1. Identificazione - Ad ogni step t, l'algoritmo osserva l'insieme S_t e deve mandare in output un indice i, che il suo tentativo di indovinare il linguaggio K.
   Si dirà che l'algoritmo **Identifica K nel limite** se $\exists t^*$ t.c. l'algoritmo manda in output l'indice di $K \forall t \geq t^*$.
2. Generazione - Ad ogni step t l'algoritmo osserva l'insieme S_t e manda in un output una stringa a_t che è il suo tentativo di indovinare una stringa mai vista di K.
   Si dirà che l'algoritmo **Genera da K nel limite** se $\exists t^*$ t.c. $a_t \in K \backslash S_t \quad \forall t \geq t^*$.

Il fatto che i linguaggi considerati nell'insieme C siano infiniti è un'importante differenza rispetto al modello originale di Gold-Angluin. Questo è necessario nel modello perché l'obiettivo è quello di stampare, nel caso della generazioene, infiniti elementi di un linguaggio fissato non ancora enunciatii. Ciò porta ovviamente ad avere un insieme dei possibili output vuoto nel caso in cui il linguaggio fissato sia finito, in quanto esso verrebbe esaurito nell'enumerazione.

La seconda importante osservazione è che l'algoritmo non ha feedback né nel caso della generazione, né in quello dell'identificazione. Quindi non sa né se le sue guess sull'indice i sono corrette, né se le stringhe in output appartengono o meno all'insieme designato.

1.3 Risultati

Sappiamo dai risultati degli studi di Gold-Angluin che un algoritmo di questo tipo non riesce ad identificare K. Ed è anche per questo motivo che i seguenti risultati della ricerca sono così sbalorditivi.

Theorem 1. *Sia data una collezione enumerabile di linguaggi $C = \{L_1, L_2, \dots\}$. Sia fissato ed enumerato uno di questi linguaggi K.*

Allora esiste una funzione $f_C : \mathbb{P}(U) \to U$. con la proprietà che $\exists\ t^*$ t.c. $f_C(S_t) \in K \backslash S_t \forall\ t \geq t^*$.

Theorem 2. *Sia data una collezione enumerabile di linguaggi $C = \{L_1, L_2, \dots\}$. Sia fissato ed enumerato uno di questi linguaggi K. Allora esiste un algoritmo che genera da K nel limite.*

Chapter 2

Identificazione nel limite

In questa sezione vediamo un caso in cui l'identificazione fallisce.
Sia Σ^* l'insieme dei numeri interi.

Siano $a \in \mathbb{Z},\ b \in \mathbb{N}$, definisco le due progressioni artimetiche (Linguaggi)

- $P_{a,b} := \{a + ib \mid i \in \mathbb{Z}_{\geq 0}\}$
- $Q_{a,b} := \{a + ib \mid i \in \mathbb{Z}\}$

Sia C la collezione di tutti i linguaggi $P_{a,b},\ Q_{a,b}$.

Supponiamo per assurdo che esista un algoritmo in grado di identificare un linguaggio fissato $K \in C$.
Costruisco una enumerazione di K che fa fallire l'identificazione.

In particolare scelgo $K = Q_{0,1}$

Dati due interi $a, b \in \mathbb{Z}$ definisco $I[a, b]$ l'intervallo di tutti gli interi $a \leq n \leq b$.
Enumero gli elementi di Σ^* in vari stage s.

1. Per induzione lo stage s ha enumerato l'intervallo $I[-s, j(s)]$ per qualche $j(s) \geq 0$.
2. Nello stage $s + 1$ vengono enumerati altri valori così da arrivare ad enumerare $I[-(s + 1), j(s + 1)]$.
   In particolare vengono enumerati prima $-(s + 1)$ e $j(s) + 1$ e poi $j(s) + 2,\ j(s) + 3,\ \ldots$ fino ad arrivare a j(s+1).

   Ad un certo punto l'algoritmo darà come tentativo la prograssione $P_{-(s+1),1}$.
   A questo punto definiamo $j(s + 1)$ come l'intero più grande generato nello stage $s + 1$
3. Comincia lo stage $s + 2$ in modo analogo.

Dunque durante questa enumerazione di $Q_{0,1}$ l'algoritmo contninuerà a dare proposte per K della forma $P_{-j,0}$ e quindi proporrà mai $Q_{0,1}$ nel limite.

Chapter 3

Generazione nel Limite attraverso una funzione